

M5 Math Note

Chung-En (Johnny) Yu

Department of Intelligent Systems and Robotics, University of West Florida

Tip

This math note serves as supplementary content for the lecture. Students are encouraged to read it in conjunction with the lecture.

Dimensionality Reduction

- Dimensionality Reduction are the techniques for reducing features in high-dimensional datasets while retaining relevant information.
- Useful in many machine learning tasks to avoid overfitting or poor generalization.
- Simplify data, speed up computation time, lower memory usage, and gain insight into data structure.

Topics covered in this note:

- Unsupervised dimensionality reduction via PCA
- Supervised data compression via LDA

Principal Component Analysis (PCA)

[Source: GeeksForGeeks](#)

[Source: Medium](#)

Overview

Principal Component Analysis (PCA) is an unsupervised linear transformation technique widely used for:

- Dimensionality reduction and feature extraction.
- Exploratory data analysis.
- Signal denoising and other applications like stock market and bioinformatics.

Key Concepts

- **Feature Extraction vs. Feature Selection:** PCA transforms data onto a new lower-dimensional subspace while retaining most information. A new set of features are extracted from the original features which are quite dissimilar in nature (covariance between the new features is 0).

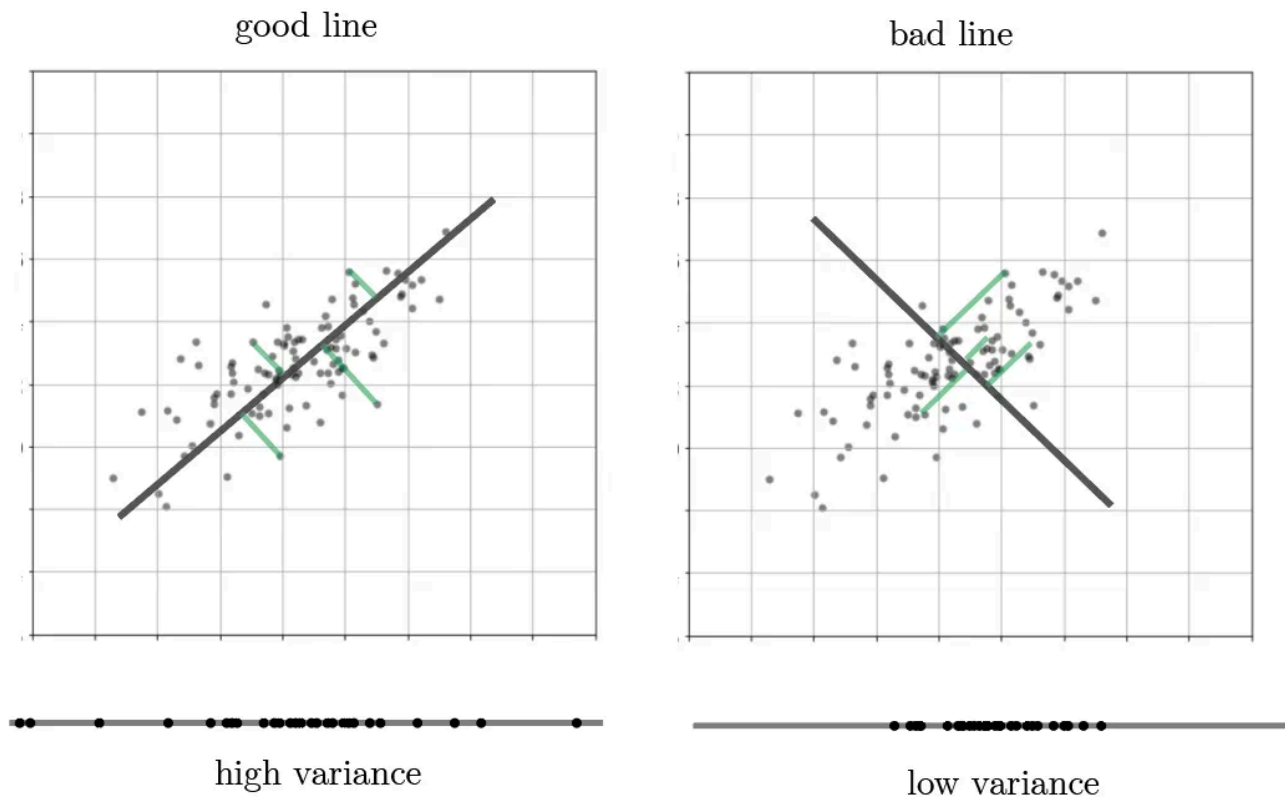
- **Variance Maximization:** PCA finds directions (principal components) with maximum variance, ensuring orthogonality between components.
- **Sensitivity to Scaling:** Features should be standardized to assign equal importance.

Key Takeaways

1. PCA transforms a dataset onto a lower-dimensional space while maximizing variance.
2. Use PCA to:
 - Reduce storage/computation costs.
 - Improve model performance by mitigating the curse of dimensionality.

Visualization

A good line will have *maximized* variance — it best preserves the locations of each point. With high variance, we see that our one-dimensional number line quite effectively retains information, showing a more complete distribution of points. We are trying to find the vector (line) through our dataset that maximizes the variance.



Math

Concept of Orthogonality

Vector Space is a set of vectors. They can be represented as a linear combination of the smaller set of vectors called **Basis Vector**. So any vector v in a vector space can be represented as:

$$v = \sum_{i=1}^n a_i u_i$$

Where,

- a represents n scalars

- u represents the basis vectors. Basis vectors are orthogonal to each other.

Orthogonality of vectors can be thought of an extension of the vectors being perpendicular in a 2D vector space. So our feature vector (dataset) can be transformed into a set of principal components (just like the basis vectors).

PCA Workflow

- First, calculate the covariance matrix of a data set.
- Then, calculate the eigenvectors of the covariance matrix.
- The eigenvector having the highest eigenvalue represents the direction in which there is the highest variance. So this will help in identifying the first principal component.
- The eigenvector having the next highest eigenvalue represents the direction in which data has the highest remaining variance and also orthogonal to the first direction. So, this helps in identifying the second principal component.
- Like this, identify the top k eigenvectors having top k eigenvalues to get the k principal components.

PCA numerical example

Consider a dummy dataset:

Feature/Sample	1	2	3	4	5	6	7	8	9	10
x_1	2.5	0.5	2.2	1.9	3.1	2.3	2.0	1.0	1.5	1.1
x_2	2.4	0.7	2.9	2.2	3.0	2.7	1.6	1.1	1.6	0.9

Step 1. Standardize the dataset

Scale data to zero mean and unit variance via `StandardScaler()`.

- Mean of x_1 , $\mu_1 = 1.81$.
- Mean of x_2 , $\mu_2 = 1.91$.

After standardization:

Feature/Sample'	1	2	3	4	5	6	7	8	9	10
x_1	0.69	-1.31	0.39	0.09	1.29	0.49	0.19	-0.81	-0.31	-0.71
x_2	0.49	-1.21	0.99	0.29	1.09	0.79	-0.31	-0.81	-0.31	-1.01

Step 2. Construct Covariance Matrix

Calculate pairwise feature covariances. The covariance matrix represents pairwise covariances between features. For two features x_j and x_k , the covariance is:

$$\sigma_{jk} = \frac{1}{n} \sum_{i=1}^n \left(x_j^{(i)} - \mu_j \right) \left(x_k^{(i)} - \mu_k \right)$$

- n : Number of samples.

- μ_j : Mean of feature j , vice versa for μ_k .
 - For a standardized dataset (zero mean and unit variance), $\mu_j = 0$.

For example, the covariance matrix of two features can then be written as follows:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{bmatrix}$$

Another common way to express:

$$\text{Correlation Matrix} = \Sigma = \left(\frac{X^T \cdot X}{n-1} \right) = \begin{bmatrix} 0.616556 & 0.615444 \\ 0.615444 & 0.716556 \end{bmatrix}$$

Where, X is the Dataset Matrix. (In this numerical, it is a 10×2 matrix)

Step 3. Eigen Decomposition

Decompose the covariance matrix into eigenvectors (directions) and eigenvalues (magnitudes). The eigenvectors v and eigenvalues λ of the covariance matrix are defined as:

$$\Sigma v = \lambda v$$

- **Eigenvalues (λ):** Represent variance captured by each principal component.
- **Eigenvectors (v):** Represent directions of maximum variance (principal components).

Let's looking for eigenvalues first. Using the equation, $|\Sigma - \lambda \cdot I| = 0$, where λ is the eigenvalue and I is the identity matrix. Now we can obtain λ :

$$\begin{aligned} & \begin{bmatrix} 0.616556 & 0.615444 \\ 0.615444 & 0.716556 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ & \begin{bmatrix} 0.616556 - \lambda & 0.615444 \\ 0.615444 & 0.716556 - \lambda \end{bmatrix} = 0 \\ & 0.44180 - 0.616556\lambda - 0.716556\lambda + \lambda^2 - 0.37877 = 0 \\ & \lambda^2 - 1.33311\lambda + 0.06303 = 0 \end{aligned}$$

Thus, we get $\lambda_1 = 1.28403$ and $\lambda_2 = 0.0490834$.

Then, we can also get eigenvectors for each eigenvalue:

$$v_1 = \begin{bmatrix} 0.67787 \\ 0.73518 \end{bmatrix} \quad v_2 = \begin{bmatrix} 0.735179 \\ -0.677873 \end{bmatrix}$$

Step 4. Sorting Eigenvalues

To reduce dimensionality, sort eigenvalues in descending order:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$$

Select the top k eigenvalues and their corresponding eigenvectors for the k -dimensional subspace. The eigenvector with the highest eigenvalue is the Principal Component of the dataset.

Step 5. Construct Projection Matrix

Select top eigenvectors and project data onto them. Construct the projection matrix W using the top k eigenvectors:

$$W = [v_1, v_2, \dots, v_k] = \begin{bmatrix} 0.677873 & 0.735179 \\ 0.735179 & -0.677879 \end{bmatrix}$$

Here, W maps the original d -dimensional feature space to the new k -dimensional space.

Step 6. Data Transformation

Transform the dataset X (size $n \times d$) to the new subspace:

$$Z = XW$$

- Z : Transformed data in the lower-dimensional k -dimensional subspace.
- Note that X is standardized dataset matrix, not original dataset.

In numerical example,

$$Z = \begin{bmatrix} 0.69 & 0.49 \\ -1.31 & -1.21 \\ 0.39 & 0.99 \\ 0.09 & 0.29 \\ 1.29 & 1.09 \\ 0.49 & 0.79 \\ 0.19 & -0.31 \\ -0.81 & -0.81 \\ -0.31 & -0.31 \\ -0.71 & -1.01 \end{bmatrix} \cdot \begin{bmatrix} 0.677873 & 0.735179 \\ 0.735179 & -0.677879 \end{bmatrix} = \begin{bmatrix} 0.8297008 & 0.17511574 \\ -1.77758022 & -0.14285816 \\ 0.99219768 & -0.38437446 \\ 0.27421048 & -0.13041706 \\ 1.67580128 & 0.20949934 \\ 0.91294918 & -0.17528196 \\ -1.14457212 & -0.04641786 \\ -0.43804612 & -0.01776486 \\ -1.22382.62 & 0.16267464 \end{bmatrix}$$

Step 7. Explained Variance Ratio

Quantify the variance captured by each principal component:

$$\text{Explained Variance Ratio} = \frac{\lambda_j}{\sum_{i=1}^d \lambda_i}$$

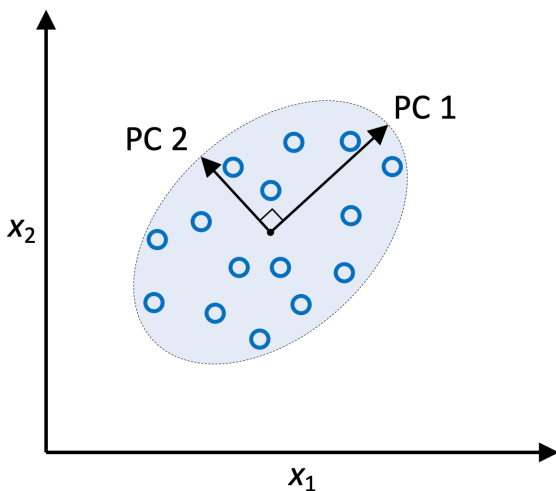
The cumulative explained variance helps determine the number of principal components to retain.

Step 8. Loadings (Feature Contributions)

The contribution of each original feature to a principal component is measured using **loadings**:

$$\text{Loadings} = v \cdot \sqrt{\lambda}$$

- Correlation between original features and principal components.



Linear Discriminant Analysis (LDA)

Overview

Linear Discriminant Analysis (LDA) is a supervised dimensionality reduction technique that transforms the feature space to optimize class separability. It achieves this by maximizing the class-discriminatory information while minimizing within-class variability.

Unlike PCA, which is unsupervised and focuses on maximizing variance, LDA leverages class labels to find linear combinations of features that best separate different classes.

Key Takeaways

1. LDA is most effective for supervised tasks, such as classification, where class separability is critical.
2. Assumes:
 - Normally distributed data.
 - Equal covariance matrices for all classes.
 - Statistical independence of training examples.
3. Produces at most $c - 1$ linear discriminants, where c is the number of classes.

Steps in LDA

Step 1. Standardize the d -dimensional dataset (d is the number of features).

Step 2. For each class, compute the d -dimensional mean vector. Compute the mean vector m_i for each class i , which stores the mean feature value, μ_m :

$$\mathbf{m}_i = \frac{1}{n_i} \sum_{x \in D_i} x_m$$

- D_i : Set of samples in class i .
- n_i : Number of samples in class i .
- μ_m : Mean feature value for examples of class i :

$$\mathbf{m}_i = \begin{bmatrix} \mu_{i, \text{feature 1}} \\ \mu_{i, \text{feature 2}} \\ \vdots \\ \mu_{i, \text{feature 3}} \end{bmatrix}^T$$

Step 3. Construct the between-class scatter matrix, S_B , and the within-class scatter matrix, S_W .

Within-Class Scatter Matrix (S_W) Measures the spread of data within each class:

$$S_W = \sum_{i=1}^c S_i, \quad S_i = \sum_{x \in D_i} (\mathbf{x} - \mathbf{m}_i)(\mathbf{x} - \mathbf{m}_i)^T$$

3.b. Between-Class Scatter Matrix (S_B) Measures the spread of the class means relative to the overall mean:

$$S_B = \sum_{i=1}^c n_i (\mathbf{m}_i - \mathbf{m})(\mathbf{m}_i - \mathbf{m})^T$$

Step 4. Compute the eigenvectors and corresponding eigenvalues of the matrix, $S_W^{-1}S_B$. Solve the eigenvalue problem for $S_W^{-1}S_B$:

$$S_W^{-1}S_B v = \lambda v$$

- **Eigenvalues (λ):** Class separability.
- **Eigenvectors (v):** Directions maximizing class separability.

Step 5. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors. Sort eigenvalues ($\lambda_1, \lambda_2, \dots$) in descending order:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{c-1}$$

Step 6. Choose the k eigenvectors that correspond to the k largest eigenvalues to construct a $d \times k$ -dimensional transformation matrix, W ; the eigenvectors are the columns of this matrix. Select the top k eigenvalues and their corresponding eigenvectors to form the $d \times k$ projection matrix, $W = [v_1, v_2, \dots, v_k]$.

Step 7. Project the examples onto the new feature subspace using the transformation matrix, W . Transform the dataset X (size $n \times d$) into the new k -dimensional subspace:

$$Z = XW$$
